INFORME TÉCNICO FINAL DE REQUERIMIENTOS

Sistema de Gestión de Restaurante

Arquitectura MVC · Python · CustomTkinter · SQLAlchemy · SQLite

Sistema:	Gestión de Restaurante
Framework GUI:	CustomTkinter 5.2.2
ORM / BD:	SQLite
Arquitectura:	MVC (Modelo-Vista-Controlador)
Repositorio:	GitHub — Alexander200318
Integrantes	Adriana Cornejo Ulloa, Ismael Loja, Victor Mendez, Alfonso Chevez
Año:	2026

1. Introducción

El presente informe describe los requerimientos funcionales y no funcionales del sistema de gestión de restaurante desarrollado como proyecto académico en el ámbito del desarrollo de software. Este sistema tiene como objetivo principal optimizar la administración de mesas, pedidos, pagos e inventario de un restaurante, mediante una aplicación de escritorio con interfaz gráfica moderna, persistencia de datos mediante ORM y arquitectura modular.

El proyecto fue desarrollado utilizando Python 3.8+ con la biblioteca CustomTkinter para la interfaz gráfica (tema oscuro profesional), SQLAlchemy como ORM sobre SQLite como motor de base de datos, y siguiendo estrictamente el patrón arquitectónico MVC. El código fuente se encuentra versionado en GitHub.

2. Objetivo del Sistema

Desarrollar una aplicación de escritorio que permita gestionar de manera eficiente las operaciones completas de un restaurante, cubriendo los siguientes procesos clave:

- Control y visualización visual del estado de las mesas (Libre / Ocupada / Reservada).
- Gestión completa de pedidos con estados progresivos y comanda digital a cocina.
- Administración de pagos con múltiples métodos y generación de facturas PDF.
- Control de inventario de ingredientes con descuento automático al vender platos.
- Generación de reportes gráficos de ventas y análisis de popularidad de platos.
- Gestión de turnos del personal de servicio con registro de asistencia.
- Administración de clientes, empleados y menú del día desde interfaz centralizada.

3. Stack Tecnológico

El sistema utiliza un stack moderno 100% en Python, con separación clara entre capa visual, lógica de negocio y persistencia de datos.

3.1 Frontend — Interfaz Gráfica

Tecnología	Versión	Propósito
CustomTkinter	5.2.2	Framework GUI moderno sobre Tkinter con tema dark/light personalizable
Pillow	10.1.0	Procesamiento e inserción de imágenes en la interfaz
Matplotlib	Latest	Generación de gráficos y reportes visuales (pie charts, bar charts)
NumPy	Latest	Operaciones numéricas y análisis estadístico de datos
tkCalendar	Latest	Widget de calendario para selección de fechas en formularios
ReportLab	4.1.0	Generación de documentos PDF (facturas, reportes de ventas)

3.2 Backend — Lógica y Persistencia

Tecnología	Versión	Propósito
Python	3.8+	Lenguaje base del sistema completo
SQLAlchemy	2.0.23	ORM para definición de modelos, relaciones y consultas a BD

SQLite	Embebida	Base de datos relacional embebida (archivo restaurante.db)
PyTZ	2023.3	Manejo de zonas horarias en registros de turnos y pedidos
python-dateutil	2.8.2	Manipulación avanzada y cálculo de fechas

4. Arquitectura del Sistema

4.1 Patrón MVC (Model-View-Controller)

El sistema implementa estrictamente el patrón arquitectónico MVC con separación total de responsabilidades entre sus tres capas principales:

```
restaurante_mvc_pyqt6/
├── models/           # Lógica de dominio (sin dependencias GUI)
├── views/            # Interfaz de usuario (CustomTkinter)
│   ├── pages/        # Páginas de cada módulo
│   └── components/   # Widgets reutilizables
├── controllers/      # Lógica de negocio (capa intermedia)
├── database/         # ORM SQLAlchemy, modelos y consultas
├── utils/            # Validadores y funciones utilitarias
├── config.py         # Configuración global y enums
└── main.py           # Punto de entrada (Entry Point)
```

4.2 Flujo de Datos

El flujo de información entre capas sigue el siguiente orden, garantizando desacoplamiento entre la interfaz y la lógica de negocio:

```
Interfaz (GUI — CustomTkinter)
    ↓  Usuario interactúa con la vista
Controlador  (controllers/)
    ↓  Procesa solicitud · Valida entrada · Coordina modelos
Modelo  (models/)
    ↓  Accede a la base de datos
Base de Datos  (SQLite vía SQLAlchemy ORM)
    ↓  Recupera / Persiste datos
Modelo  →  Controlador  →  Vista  (Actualiza interfaz)
```

Todos los modelos retornan una tupla estándar de 3 elementos:

```
(success: bool, data: Any, mensaje: str)
```

```
Ejemplo:
(True, cliente_obj, 'Cliente creado exitosamente')
(False, None, 'Error: el campo email es requerido')
```

4.3 Descripción de Capas

database/ — Gestión de Base de Datos

- **db_manager.py:** DatabaseManager con Singleton pattern. Gestiona conexiones SQLAlchemy con StaticPool (thread-safe). Métodos: create_tables(), get_session(), _init_db().
- **models.py:** Definición ORM de todas las entidades: Mesa, Cliente, Pedido, DetallePedido, Plato, Ingrediente, Empleado, Turno, Pago y tabla M2M plato_ingrediente_association.
- **queries.py:** Consultas SQL reutilizables con JOIN para reportes y listados complejos.

models/ — Modelos de Dominio

- **base_model.py:** Clase base con métodos CRUD comunes para todos los modelos.
- **Módulos específicos:** clientes.py, empleados.py, ingredientes.py, mesas.py, pagos.py, pedidos.py, platos.py, turnos.py — cada uno encapsula la lógica de su entidad.

controllers/ — Lógica de Negocio

- **base_controller.py:** BaseController con Template Method para validaciones y manejo de errores comunes.
- **Responsabilidades:** Validar datos de entrada, coordinar modelos, formatear respuestas para GUI, manejo centralizado de excepciones.

views/ — Interfaz Gráfica

- **main_window.py:** Contenedor principal. Gestiona navegación entre páginas y ciclo de vida de la aplicación.
- **sidebar.py:** Barra lateral de navegación con cambio dinámico de páginas.
- **components/:** TreeViewWidget (tabla avanzada), DialogUtils (diálogos), Formatters (dinero, fechas), PagoCard, PedidoCard.
- **pages/:** clientes_page, empleados_page, ingredientes_page, mesas_page, menu_page, pedidos_page, pagos_page, dashboard, reportes_page.

4.4 Patrones de Diseño Implementados

Patrón	Ubicación	Propósito
MVC	Estructura general	Separación total de responsabilidades entre capas
Singleton	DatabaseManager	Una única instancia de conexión a la base de datos
Observer (implícito)	Views ↔ Controllers	Comunicación reactiva entre capas
Factory	Controllers	Creación estandarizada de instancias de modelos
DAO	database/	Acceso centralizado y abstraído a los datos
Template Method	BaseController, BaseModel	Métodos comunes heredados por todas las subclases

5. Esquema de Base de Datos

Base de datos: SQLite · Archivo: restaurante.db · ORM: SQLAlchemy 2.0.23 · Configuración: StaticPool (thread-safe) · check_same_thread=False

5.1 Tablas Principales

Tabla	Descripción	Campos Clave
mesas	Mesas del restaurante	id, numero, capacidad, estado
clientes	Información de clientes	id, nombre, email, telefono, mesa_id (FK)
pedidos	Pedidos realizados	id, mesa_id (FK), fecha, estado, total
detalle_pedido	Líneas de cada pedido	id, pedido_id (FK), plato_id (FK), cantidad, precio
platos	Menú disponible	id, nombre, descripcion, precio, categoria
ingredientes	Inventario de insumos	id, nombre, cantidad, unidad, precio_unitario
plato_ingrediente	Relación M2M Plato-Ingrediente	plato_id (FK), ingrediente_id (FK), cantidad_requerida
empleados	Personal del restaurante	id, nombre, puesto, salario, telefono
turnos	Turnos laborales	id, empleado_id (FK), fecha_inicio, fecha_fin, tipo
pagos	Registro de pagos	id, pedido_id (FK), monto, metodo, fecha, estado

5.2 Relaciones entre Entidades

```
Mesa (1) ──────→ (N) Cliente          [mesa_id FK en clientes]
Mesa (1) ──────→ (N) Pedido           [mesa_id FK en pedidos]
Pedido (1) ─────→ (N) DetallePedido   [pedido_id FK en detalle_pedido]
DetallePedido (N) ─→ (1) Plato        [plato_id FK en detalle_pedido]
Plato (N) ←─────→ (N) Ingrediente     [tabla M2M: plato_ingrediente]
Pedido (1) ─────→ (1) Pago            [pedido_id FK en pagos]
Empleado (1) ───→ (N) Turno           [empleado_id FK en turnos]
```

5.3 Estados del Sistema (Enums en config.py)

Entidad	Estados Posibles
Mesa (MesaEstado)	LIBRE | OCUPADA | RESERVADA
Pedido (EstadoPedido)	PENDIENTE | PREPARANDO | LISTO | ENTREGADO
Pago (EstadoPago)	PENDIENTE | COMPLETADO | CANCELADO

6. Requerimientos Comunes del Sistema

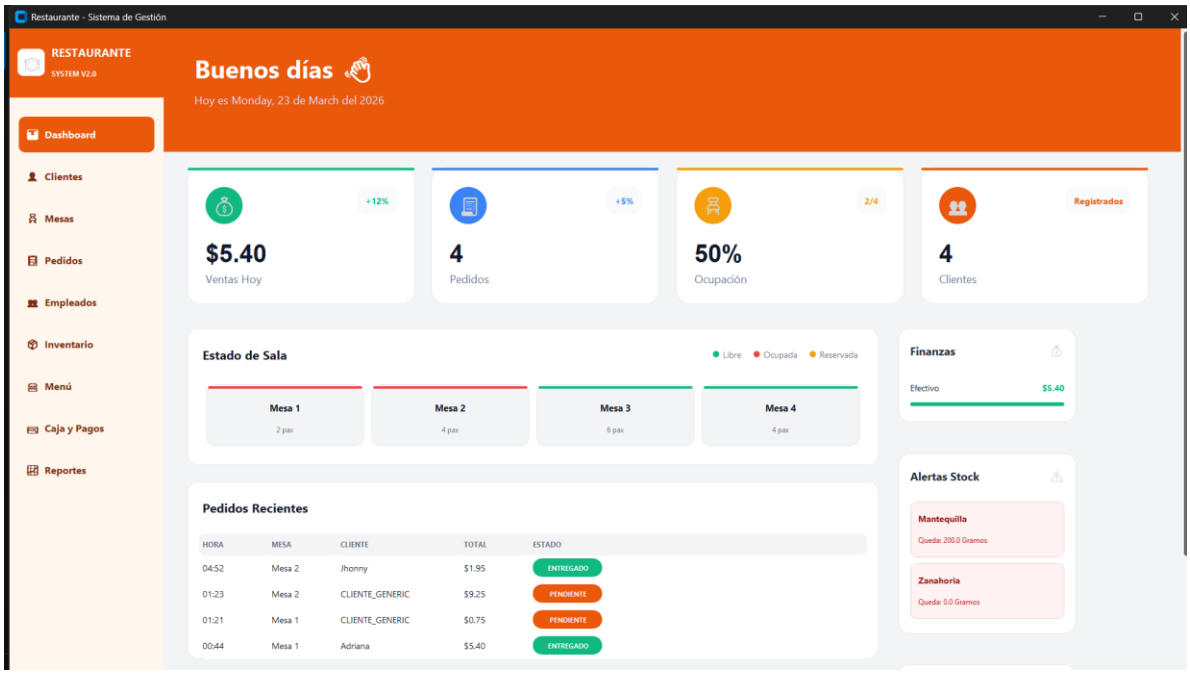
Los siguientes requerimientos son transversales a toda la aplicación y garantizan su correcto funcionamiento, usabilidad, mantenibilidad y estabilidad.

N°	Requerimiento	Descripción / Implementación
1	**Interfaz Gráfica (GUI)**	Desarrollada con CustomTkinter 5.2.2. Tema oscuro (dark mode) con paleta de colores personalizada. Interfaz moderna y profesional.
2	**Persistencia de Datos**	SQLite mediante SQLAlchemy ORM 2.0.23. Archivo restaurante.db con conexión thread-safe (StaticPool). Gestión de sesiones con autocommit deshabilitado.
3	**Arquitectura Modular (MVC)**	Separación en models/, views/, controllers/ y database/. Cada capa es independiente y reutilizable. Patrón de respuesta estándar (success, data, mensaje).
4	**Validación Robusta**	Módulo validators.py con validación de email (RFC 5322), teléfono (10-15 dígitos), fechas y valores monetarios. try-except en todas las operaciones críticas.
5	**Navegación Dinámica**	Sidebar (views/sidebar.py) con cambio dinámico de páginas. Todos los módulos accesibles sin cerrar la aplicación.
6	**Visualización de Datos**	TreeViewWidget personalizado (views/components/treeview_widget.py) con soporte para ordenamiento, edición inline y selección múltiple.

7	**Relaciones de Base de Datos**	FOREIGN KEY en todas las relaciones. Consultas JOIN implementadas. Relación M2M entre Platos e Ingredientes vía tabla plato_ingrediente_association.
8	**Feedback al Usuario**	Módulo dialog_utils.py con diálogos de confirmación, entrada de datos y mensajes de error/éxito mediante cuadros de diálogo personalizados con CustomTkinter.
9	**Gestión de Estados**	Enums en config.py: MesaEstado (LIBRE/OCUPADA/RESERVADA), EstadoPedido (PENDIENTE/PREPARANDO/LISTO/ENTREGADO), EstadoPago (PENDIENTE/COMPLETADO/CANCELADO).
10	**Escalabilidad**	BaseController y BaseModel como clases base. Patrón Template Method para extender funcionalidades. Configuración centralizada en config.py.
11	**Integración con GitHub**	Repositorio público versionado con control de cambios, historial de commits y gestión de ramas para desarrollo colaborativo.

6.1 Interfaz Gráfica (GUI)

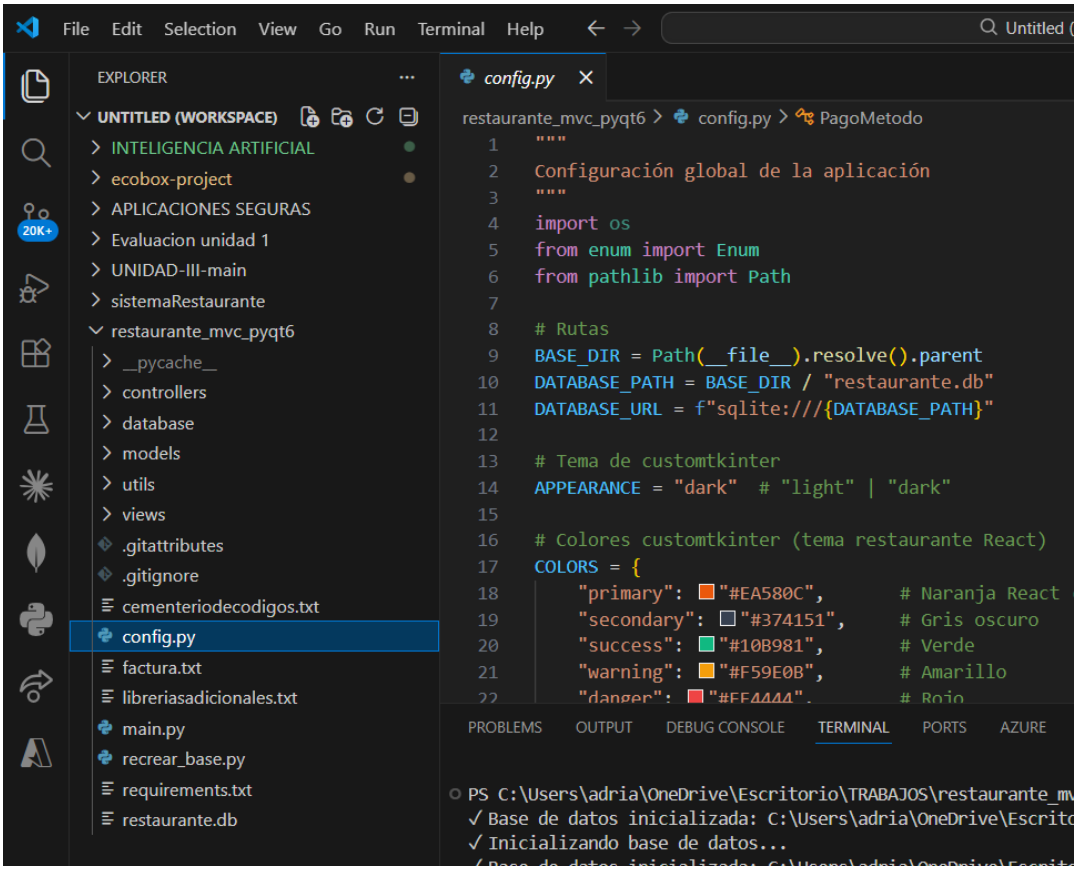
Desarrollada con CustomTkinter 5.2.2. Ofrece un tema oscuro (dark mode) con paleta de colores personalizada (color primario naranja #EA580C), ventanas con widgets modernos, formularios de entrada, tablas de datos, botones de acción y cuadros de diálogo estilizados. La apariencia es configurable desde config.py (parámetro APPEARANCE).



6.2 Persistencia de Datos

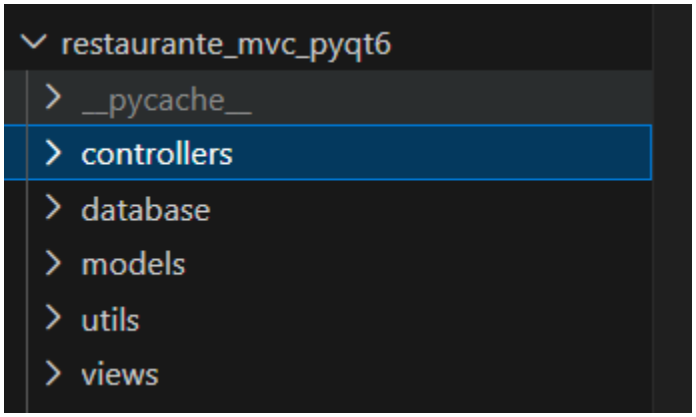
Integración con SQLite mediante SQLAlchemy ORM 2.0.23. La base de datos se almacena en el archivo restaurante.db. Las sesiones se gestionan con autocommit y autoflush deshabilitados, con

rollback automático ante errores. La configuración de conexión se centraliza en config.py (DATABASE_URL).



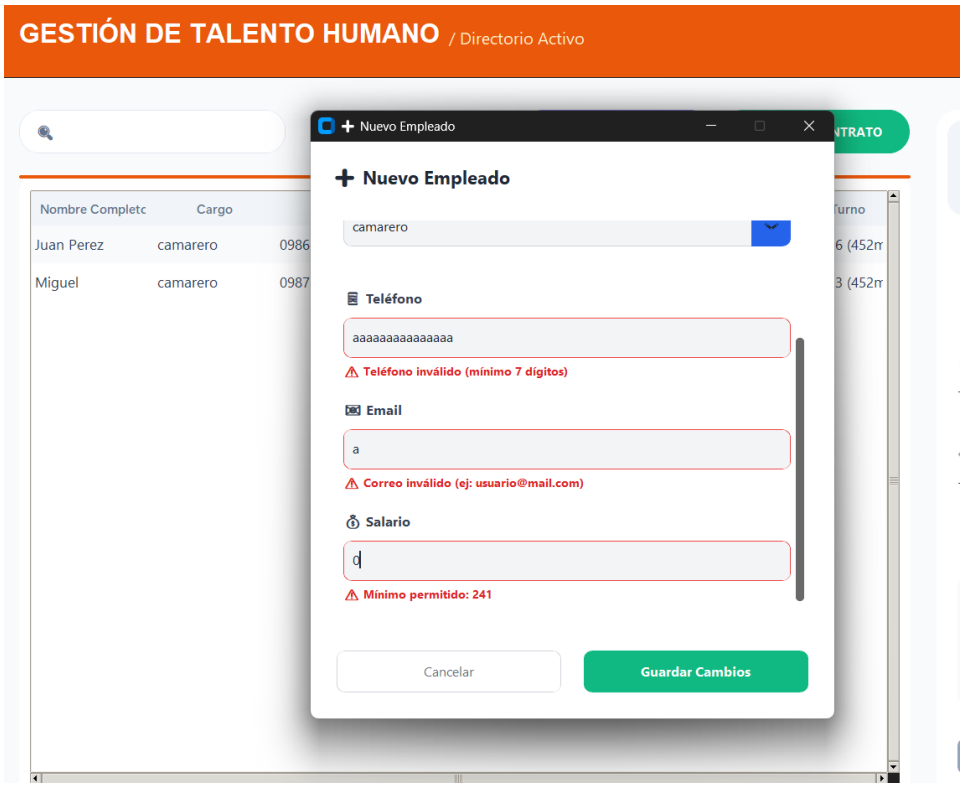
6.3 Arquitectura Modular (MVC)

Separación estricta en cuatro capas: database/ (acceso a datos con ORM), models/ (lógica de dominio), controllers/ (lógica de negocio) y views/ (interfaz gráfica). BaseController y BaseModel proveen métodos comunes mediante herencia (Template Method), facilitando la extensión del sistema.



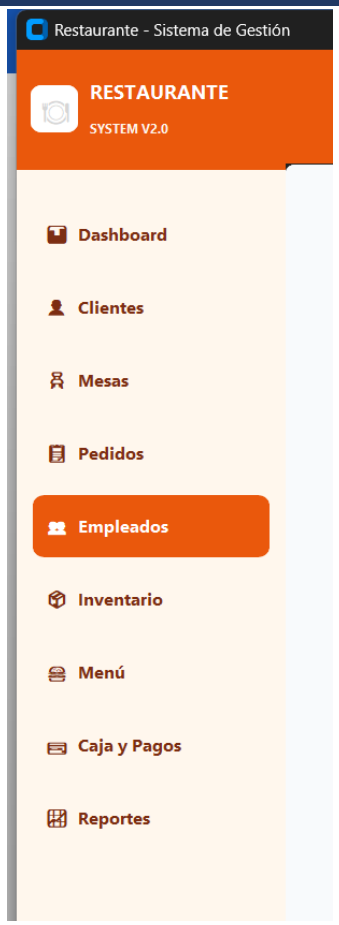
6.4 Validación Robusta

El módulo utils/validators.py implementa funciones de validación específicas: validar_email() con formato RFC 5322, validar_telefono() de 10 a 15 dígitos, validar_fecha() con verificación de rango y validar_dinero() para decimales positivos. Todas las operaciones críticas están protegidas con try-except.



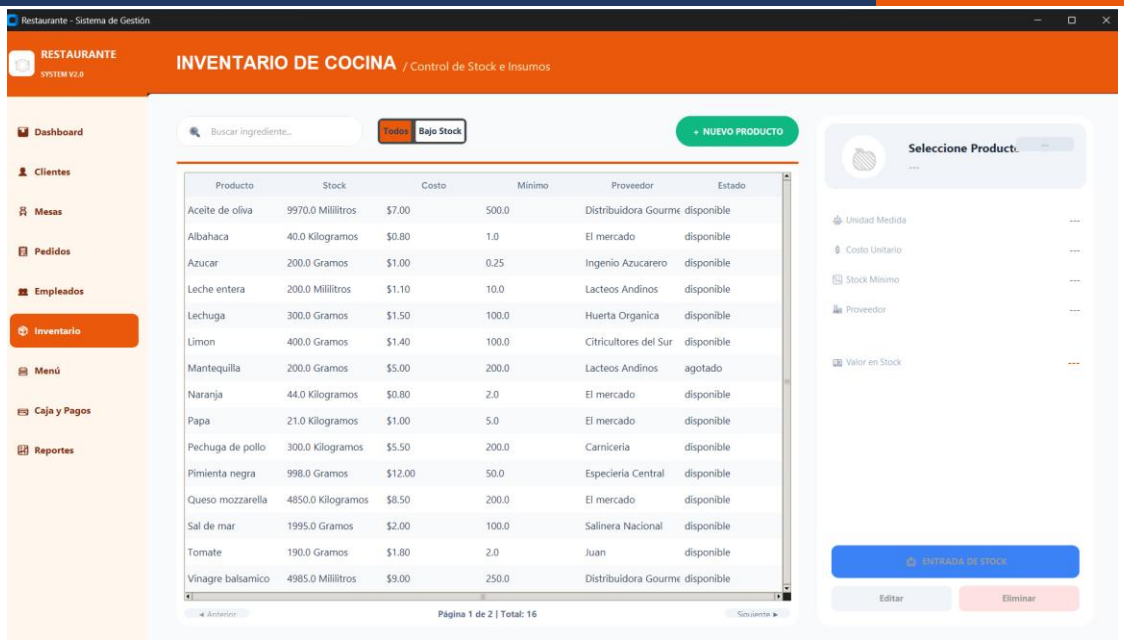
6.5 Navegación Dinámica

El componente Sidebar (views/sidebar.py) permite al usuario navegar entre los módulos del sistema (Mesas, Pedidos, Pagos, Inventario, Menú, Clientes, Empleados, Turnos, Reportes, Dashboard) sin cerrar ni reiniciar la aplicación. La ventana principal (MainWindow) gestiona el cambio dinámico de páginas.



6.6 Visualización de Datos

El componente TreeViewWidget (views/components/treeview_widget.py) provee una tabla avanzada con soporte para ordenamiento por columnas, edición inline de registros y selección múltiple. Se utiliza en todas las páginas del sistema para mostrar datos en tiempo real.



Producto	Stock	Costo	Mínimo	Proveedor	Estado
Aceite de oliva	9970.0 Mililitros	$7.00	500.0	Distribuidora Gourme	disponible
Albahaca	40.0 Kilogramos	$0.80	1.0	El mercado	disponible
Azucar	200.0 Gramos	$1.00	0.25	Ingenio Azucarero	disponible
Leche entera	200.0 Mililitros	$1.10	10.0	Lacteos Andinos	disponible
Lechuga	300.0 Gramos	$1.50	100.0	Huerta Organica	disponible
Limon	400.0 Gramos	$1.40	100.0	Citricultores del Sur	disponible
Mantequilla	200.0 Gramos	$5.00	200.0	Lacteos Andinos	agotado
Naranja	44.0 Kilogramos	$0.80	2.0	El mercado	disponible
Papa	21.0 Kilogramos	$1.00	5.0	El mercado	disponible
Pechuga de pollo	300.0 Kilogramos	$5.50	200.0	Carniceria	disponible
Pimienta negra	998.0 Gramos	$12.00	50.0	Especieria Central	disponible
Queso mozzarella	4850.0 Kilogramos	$8.50	200.0	El mercado	disponible
Sal de mar	1995.0 Gramos	$2.00	100.0	Salinera Nacional	disponible
Tomate	190.0 Gramos	$1.80	2.0	Juan	disponible
Vinagre balsamico	4985.0 Mililitros	$9.00	250.0	Distribuidora Gourme	disponible

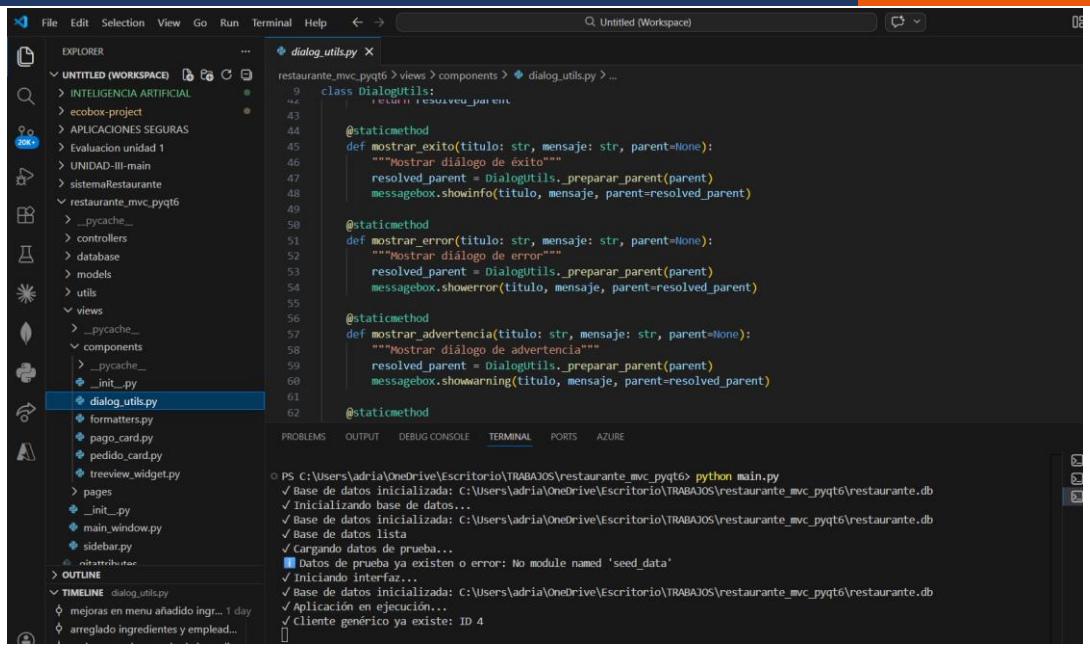
6.7 Relaciones de Base de Datos

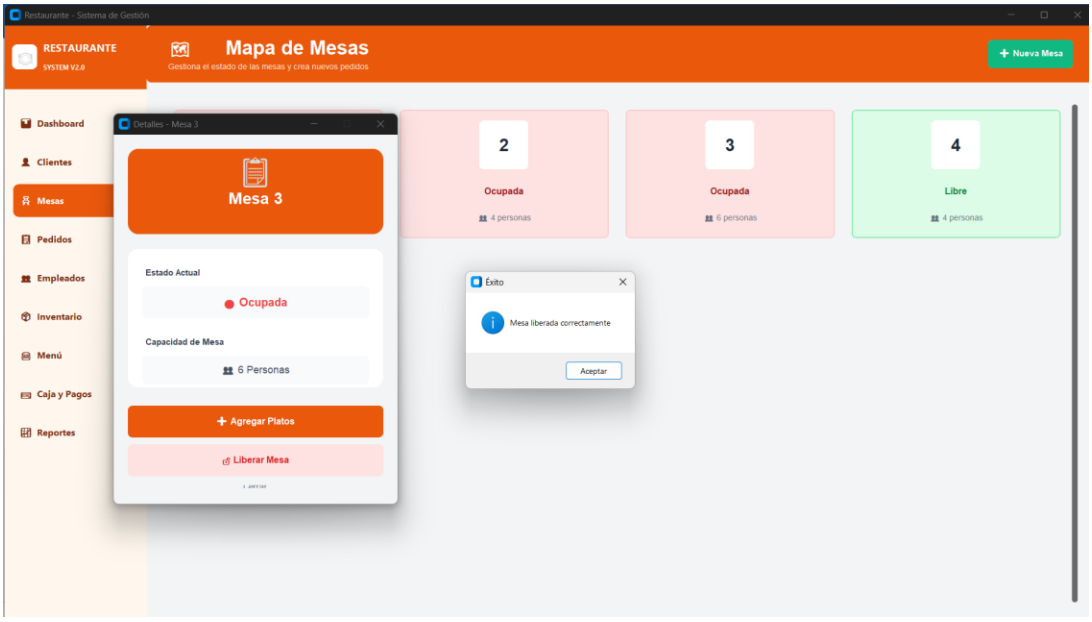
Implementación de FOREIGN KEY en todas las entidades relacionadas. Consultas con JOIN en queries.py para reportes y listados. Relación M2M entre Platos e Ingredientes mediante la tabla plato_ingrediente_association con el campo cantidad_requerida. Prevención de inyección SQL garantizada por el ORM de SQLAlchemy.

```python
class Cliente(Base):
    """Modelo de Cliente/Comensal"""
    __tablename__ = "clientes"

    id = Column(Integer, primary_key=True, index=True)
    cedula = Column(String(20), unique=True, nullable=False, index=True)
    nombre = Column(String(100), nullable=False)
    apellido = Column(String(100), nullable=False)
    correo = Column(String(100), nullable=True)
    telefono = Column(String(20), nullable=True)
    direccion = Column(String(255), nullable=True)
    mesa_id = Column(Integer, ForeignKey('mesas.id'), nullable=True)
    estado = Column(SQLEnum(config.ClienteEstado), default=config.ClienteEstado.COMIENDO)
    fecha_llegada = Column(DateTime, default=datetime.now)

    # Relaciones
    mesa = relationship("Mesa", back_populates="clientes")
    pedidos = relationship("Pedido", back_populates="cliente")
```

6.8 Feedback al Usuario

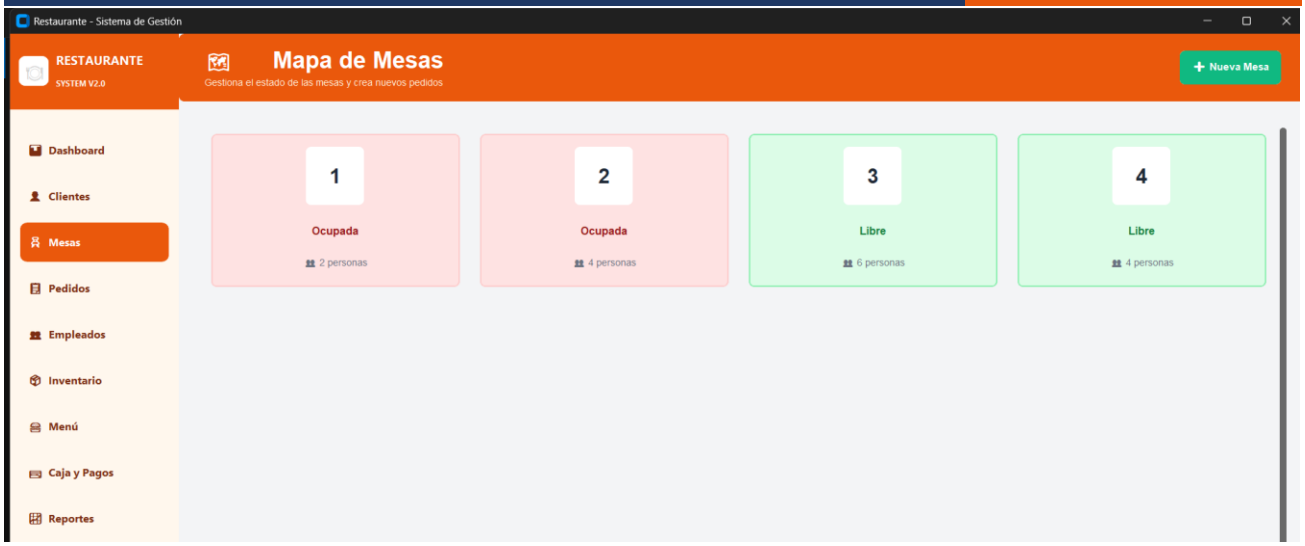
El módulo dialog_utils.py (views/components/) provee diálogos de confirmación, ventanas de entrada de datos y mensajes de error/éxito mediante cuadros de diálogo personalizados con CustomTkinter. Cada operación CRUD informa al usuario sobre su resultado mediante un mensaje claro.

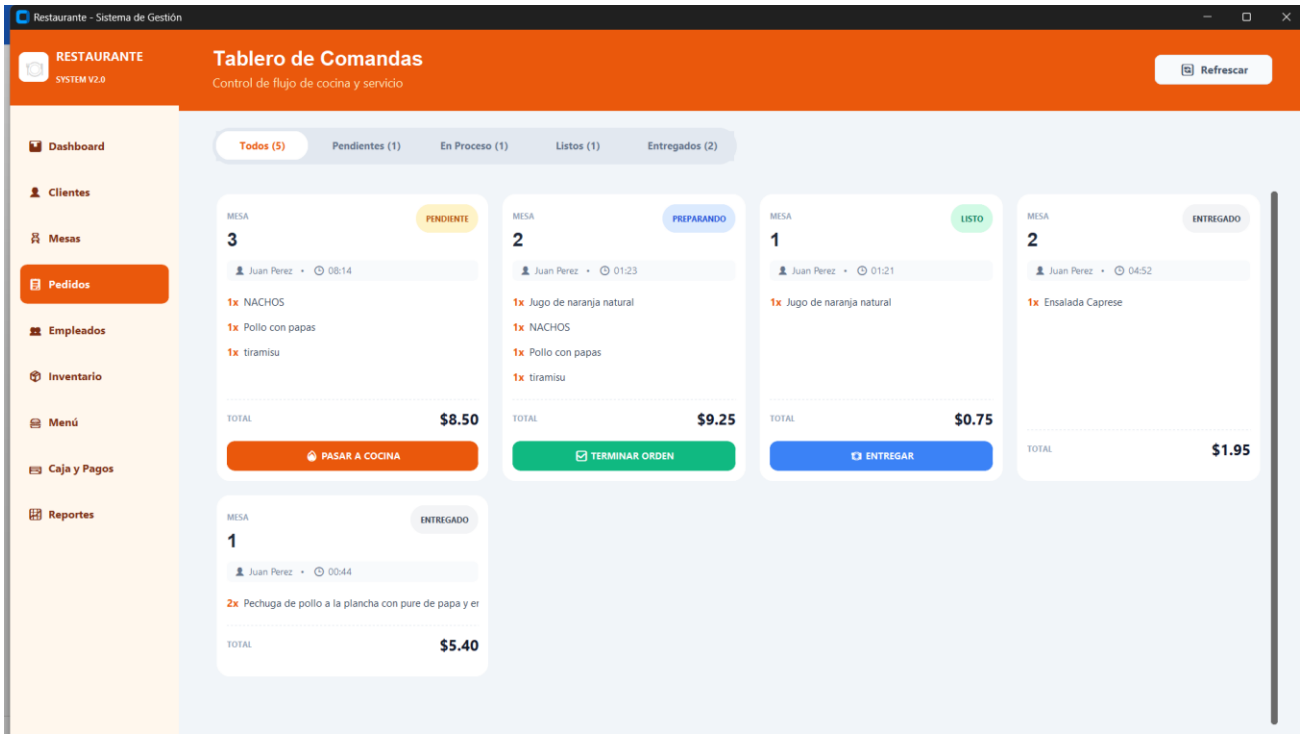




6.9 Gestión de Estados

Los estados se definen como enumeraciones (Enum) en config.py: MesaEstado (LIBRE/OCUPADA/RESERVADA), EstadoPedido (PENDIENTE/PREPARANDO/LISTO/ENTREGADO) y EstadoPago (PENDIENTE/COMPLETADO/CANCELADO). Esta implementación garantiza consistencia y facilita futuras ampliaciones.



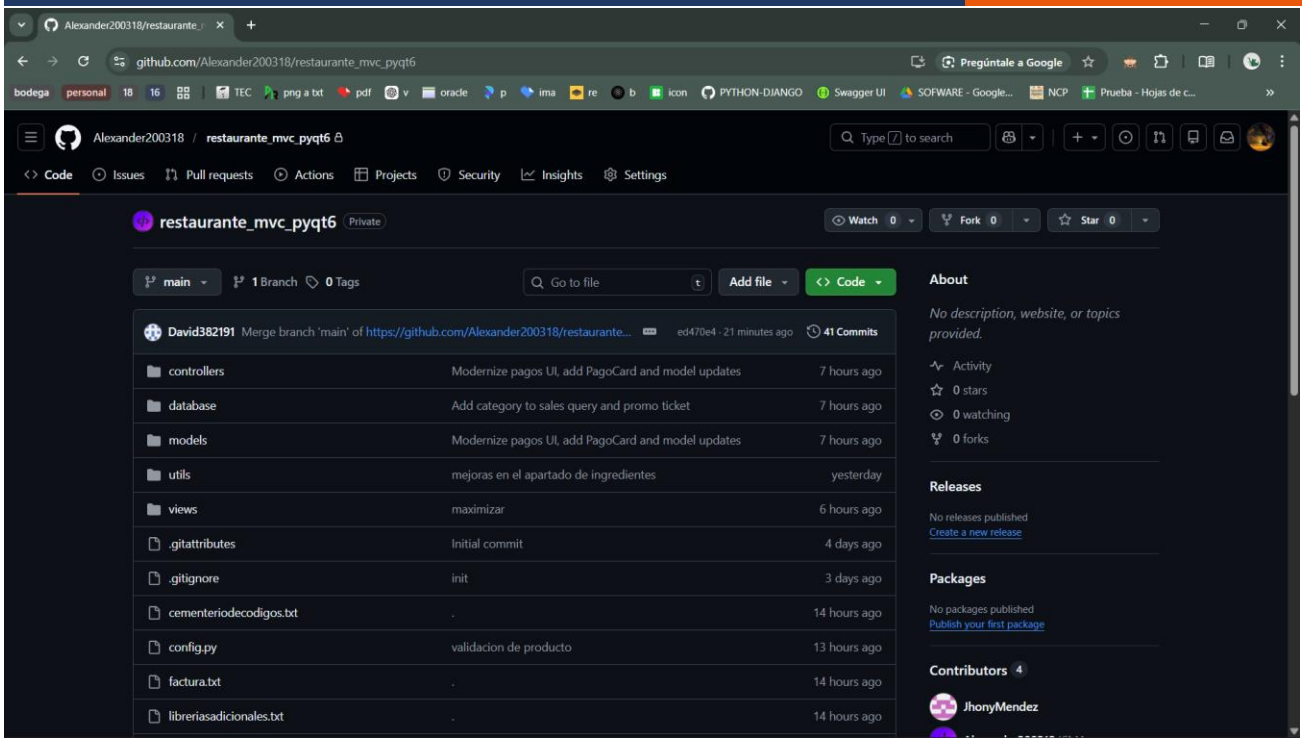


6.10 Escalabilidad

La arquitectura MVC con BaseController y BaseModel permite añadir nuevos módulos creando únicamente el modelo, controlador y vista correspondientes, sin modificar el código existente. La configuración centralizada en config.py y el sistema de colores/constantes permite adaptar el sistema rápidamente.

6.11 Integración con GitHub

Repositorio: https://github.com/Alexander200318/restaurante_mvc_pyqt6.git



7. Requerimientos Específicos — Sistema de Restaurante

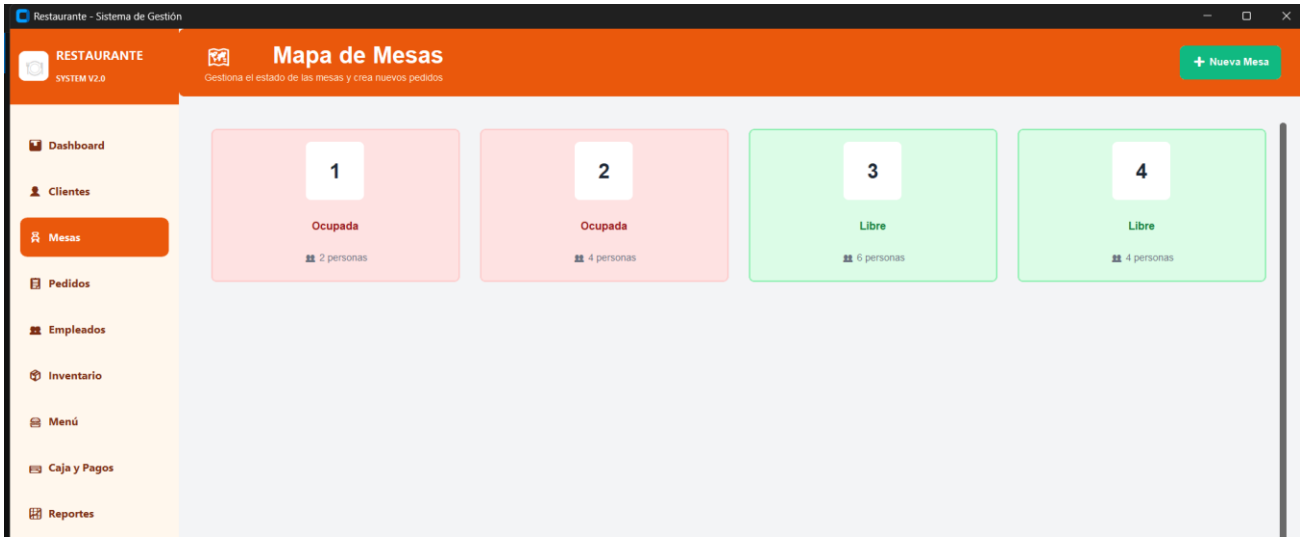
A continuación se detallan los requerimientos específicos implementados, correspondientes a las funcionalidades propias del dominio del negocio restaurantero.

N°	Requerimiento	Descripción / Implementación
1	**Mapa de Mesas Interactivo**	Visualización gráfica en mesas_page.py con estados LIBRE/OCUPADA/RESERVADA mediante código de color. Historial de ocupación incluido.
2	**Comanda Digital**	Pedidos enviados en tiempo real a vista de cocina. Estados progresivos: PENDIENTE → PREPARANDO → LISTO → ENTREGADO. Cálculo automático de total.
3	**Gestión de Ingredientes (Recetas)**	Relación M2M Plato-Ingrediente con cantidad_requerida. Descuento automático al registrar venta. Gestión de inventario en ingredientes_page.py.
4	**Múltiples Métodos de Pago**	Métodos: EFECTIVO, TARJETA, DIGITAL. Generación de factura PDF con ReportLab 4.1.0 al completar pago.
5	**División de Cuenta**	Funcionalidad en pagos_page.py para distribución del total entre N comensales con o sin propina incluida.
6	**Propina Sugerida**	Cálculo automático del 10% sobre subtotal. Modificable por el mesero antes de procesar el pago.
7	**Reporte de Platos Más Vendidos**	Gráfico circular generado con Matplotlib en reportes_page.py. Análisis por periodo con estadísticas de ventas e ingresos.

8	**Gestión de Turnos**	Módulo turnos_controller.py con registro de fecha_inicio y fecha_fin. Reportes de asistencia y horas trabajadas por empleado.
9	**Menú del Día**	Configuración rápida en menu_page.py con precio especial y disponibilidad. Platos promocionales destacados en interfaz del mesero.
10	**Impresión de Ticket**	Generación de factura PDF con ReportLab. Compatible con impresoras POS. Incluye detalle de consumo, subtotal, propina, impuestos, total y método de pago.

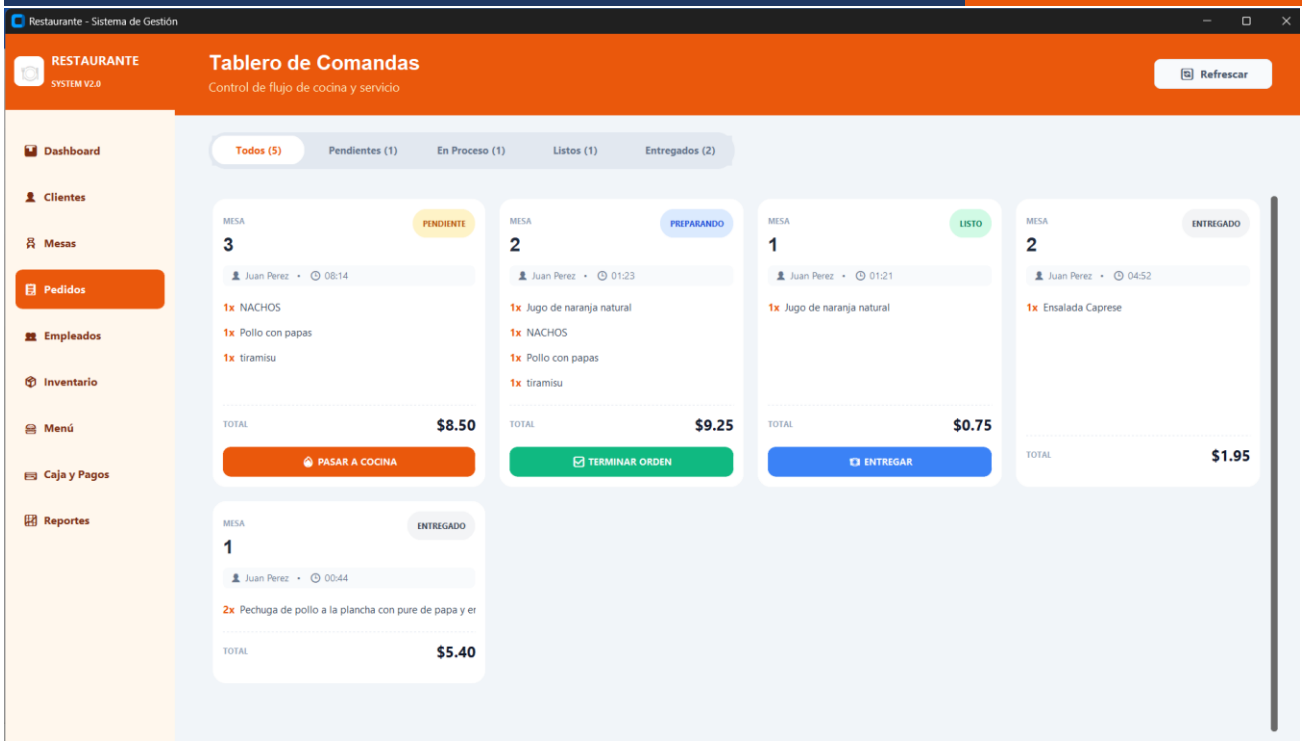
7.1 Mapa de Mesas Interactivo

La página mesas_page.py muestra una visualización gráfica de todas las mesas del restaurante. Cada mesa presenta su estado mediante código de color: verde (LIBRE), rojo (OCUPADA), amarillo (RESERVADA). Al seleccionar una mesa se accede a su historial de ocupación y pedidos activos.



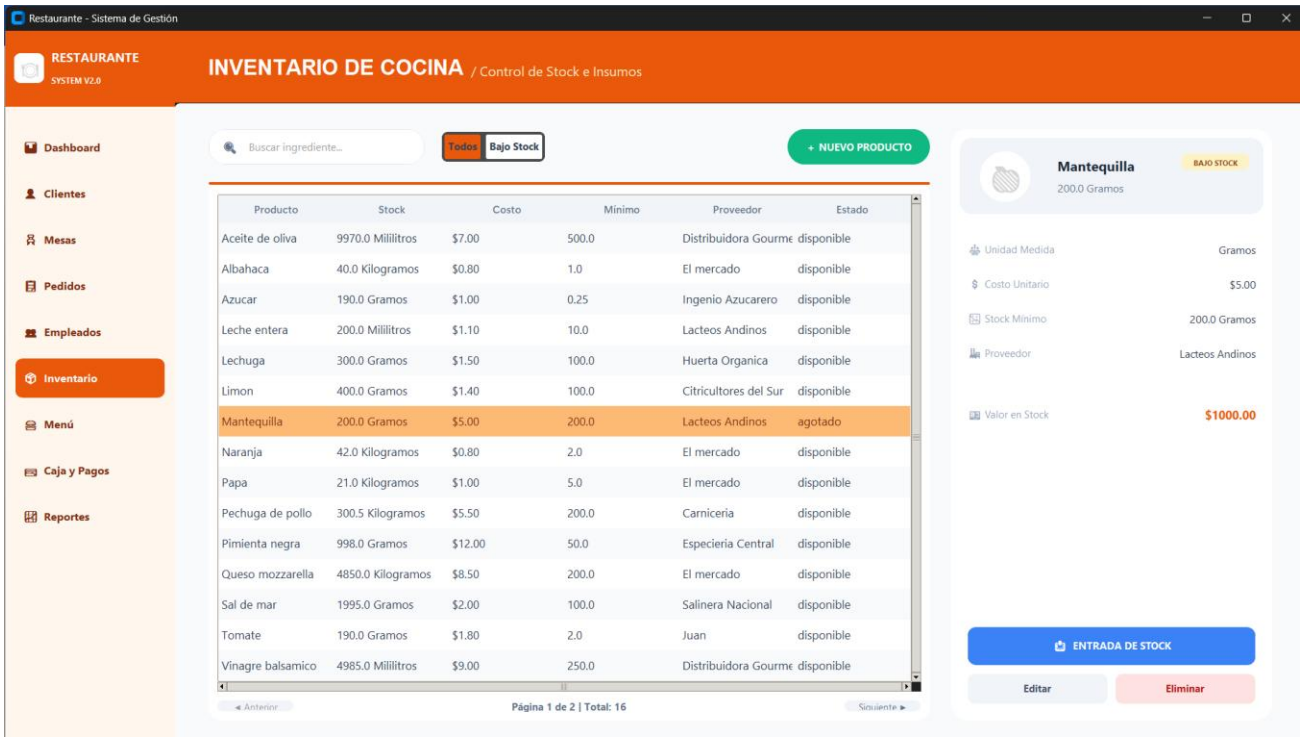
7.2 Comanda Digital

Al registrar un pedido mediante PedidosController, el sistema actualiza automáticamente la vista de cocina en tiempo real. Los pedidos muestran el número de mesa, los platos solicitados con sus cantidades y el estado progresivo: PENDIENTE → PREPARANDO → LISTO → ENTREGADO. El total se calcula automáticamente sumando precio × cantidad de cada DetallePedido.



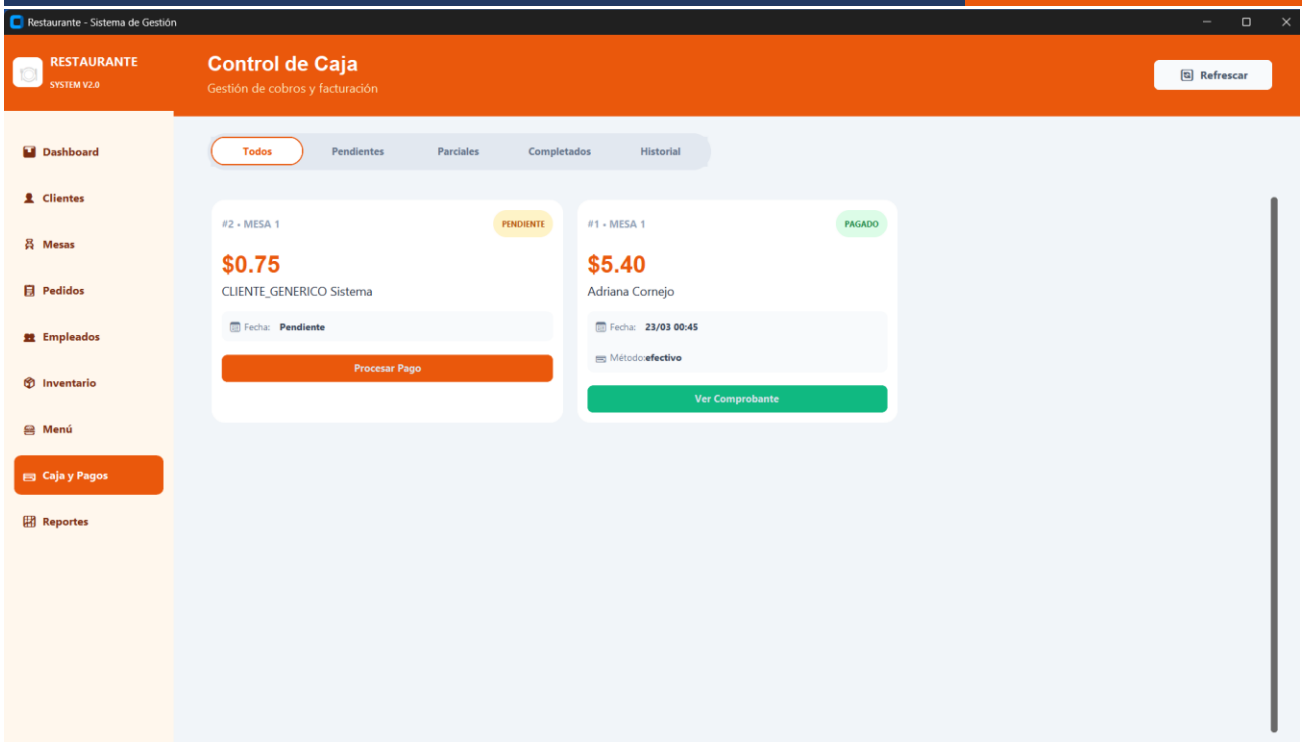
7.3 Gestión de Ingredientes y Recetas

Cada plato del menú tiene ingredientes asociados mediante la tabla M2M plato_ingrediente con el campo cantidad_requerida. Al registrar una venta, IngredientsController descuenta automáticamente del inventario las cantidades correspondientes. La página ingredientes_page.py permite gestionar stock con unidades de medida y costos unitarios.

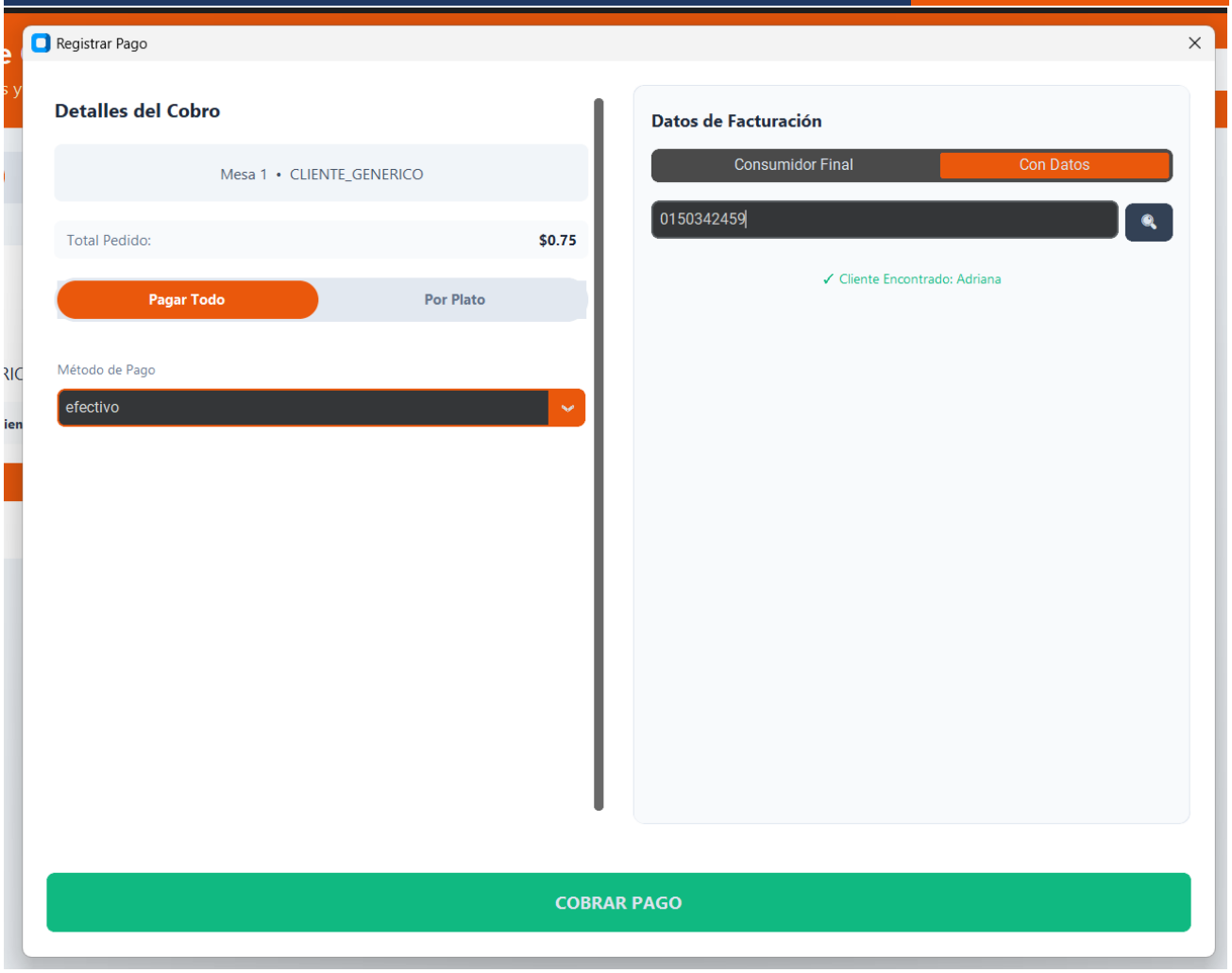


7.4 Múltiples Métodos de Pago

El módulo de pagos soporta tres métodos: EFECTIVO (con cálculo de cambio), TARJETA (con registro de número de autorización) y DIGITAL (transferencia con referencia bancaria). Al completar el pago, PagosController actualiza el estado del pedido a PAGADO y genera automáticamente la factura en formato PDF mediante ReportLab 4.1.0.

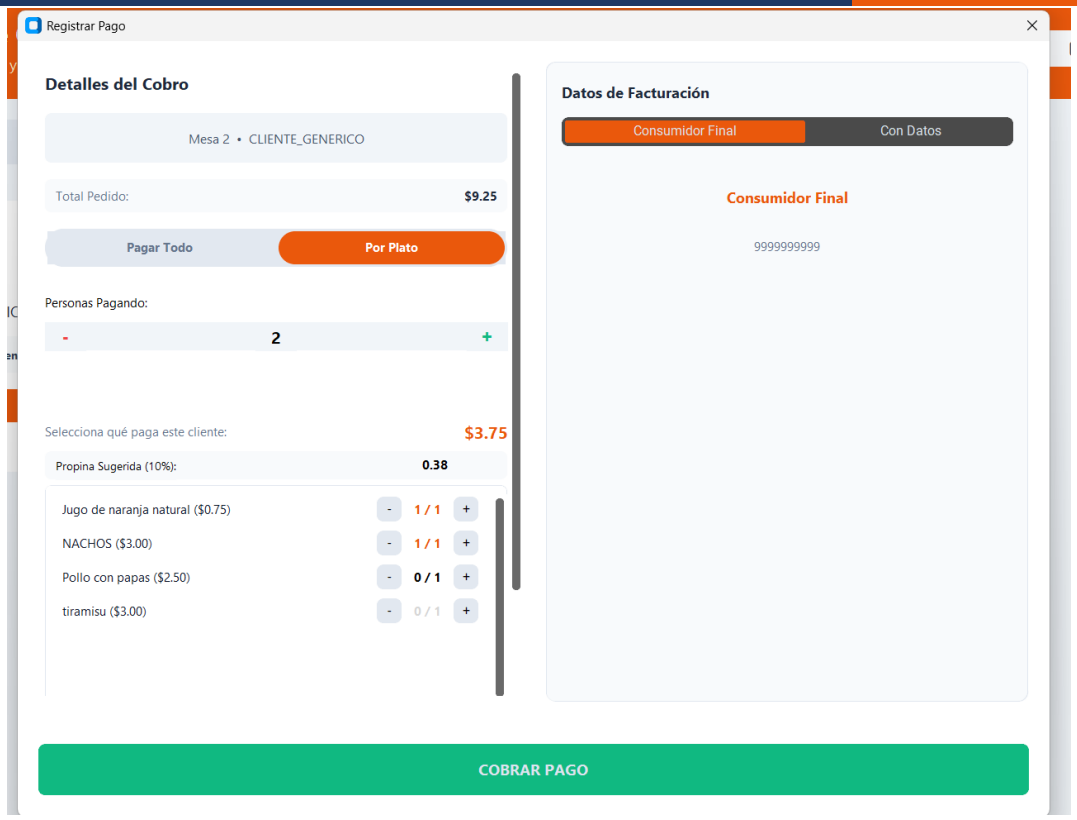

Restaurante - Sistema de Gestión
RESTAURANTE
SYSTEM V2.0
Control de Caja
Gestión de cobros y facturación
Refrescar
Dashboard
Clientes
Mesas
Pedidos
Empleados
Inventario
Menú
Caja y Pagos
Reportes
Todos
Pendientes
Parciales
Completados
Historial
#2 • MESA 1
PENDIENTE
$0.75
CLIENTE_GENERICO Sistema
Fecha: Pendiente
Procesar Pago
#1 • MESA 1
PAGADO
$5.40
Adriana Cornejo
Fecha: 23/03 00:45
Método: efectivo
Ver Comprobante

Método de Pago
efectivo
efectivo
tarjeta
transferencia
fondo_común



7.5 División de Cuenta

La funcionalidad implementada en pagos_page.py permite ingresar el número de comensales (N) para dividir el total de la mesa. El sistema calcula el valor por persona incluyendo o excluyendo la propina sugerida, facilitando el cobro individual en mesas grupales.

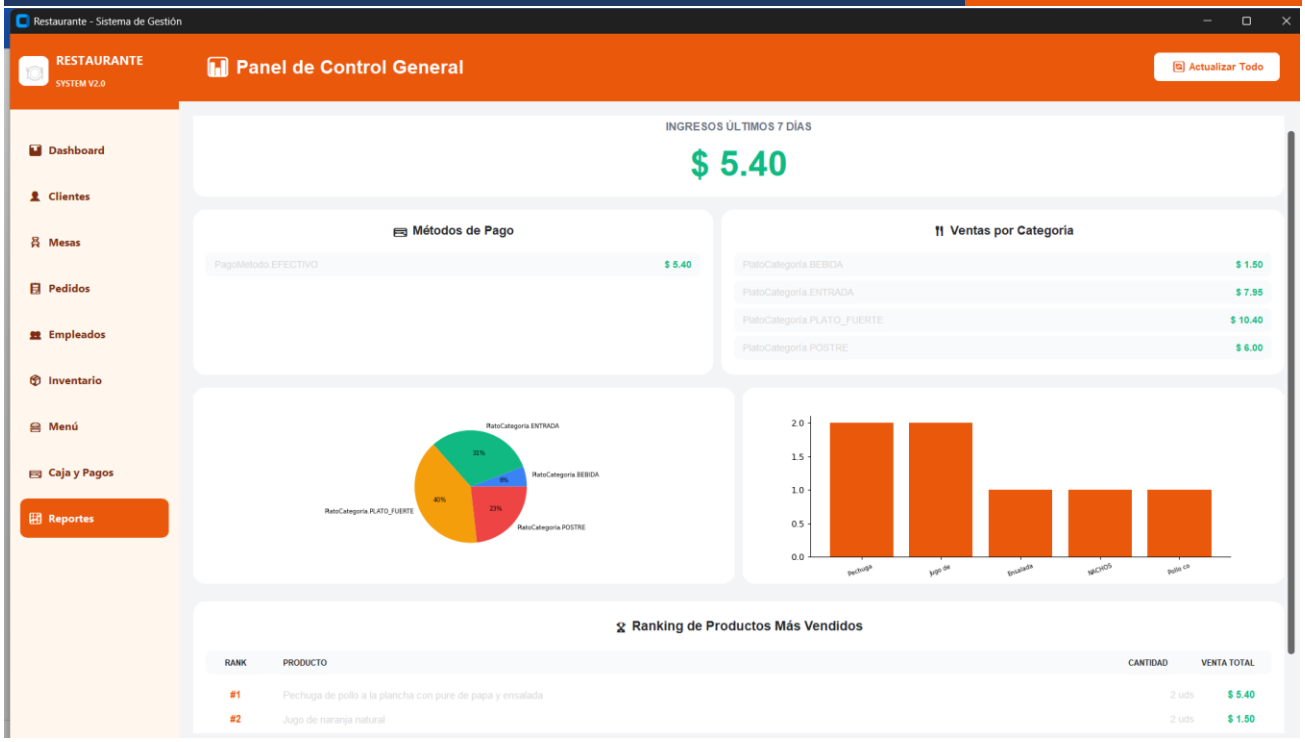


7.6 Propina Sugerida

Al iniciar el proceso de pago, el sistema muestra el cálculo automático de la propina sugerida equivalente al 10% del subtotal. El mesero puede aceptar el valor sugerido, modificarlo manualmente o eliminarlo antes de procesar el cobro final.

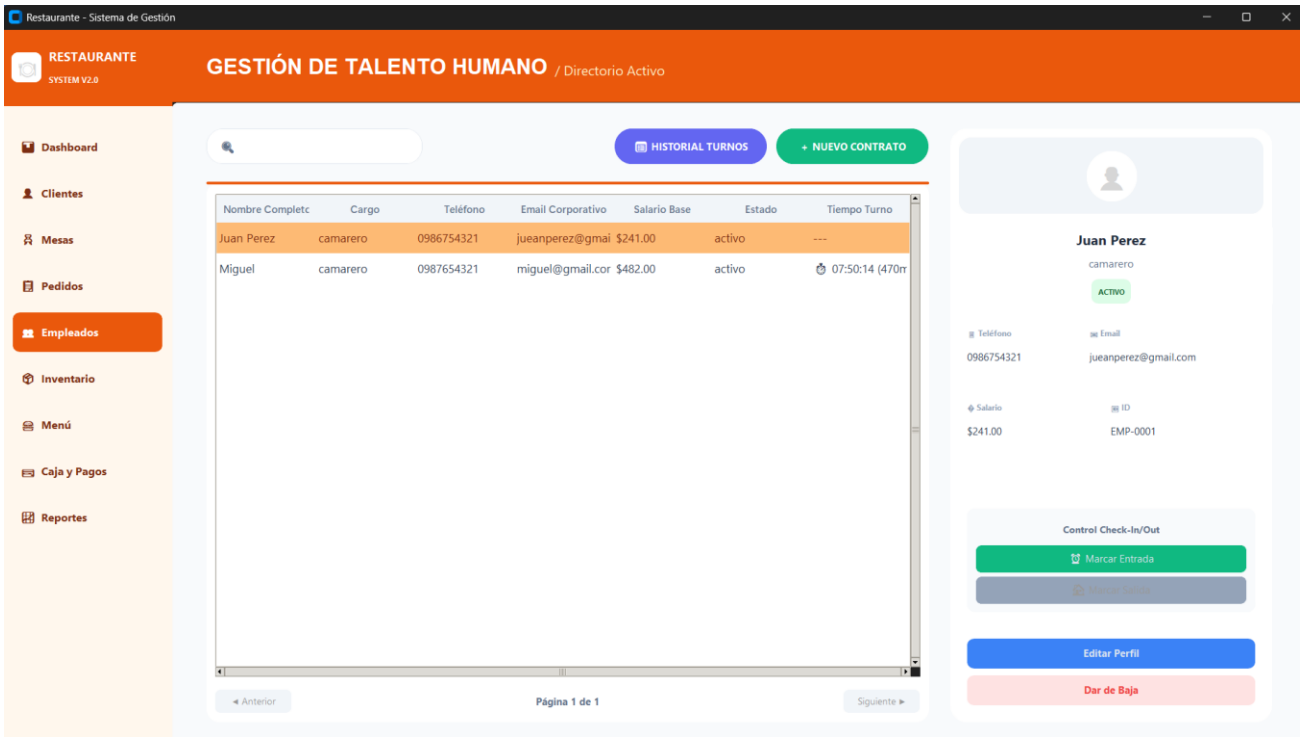
7.7 Reporte de Platos Más Vendidos

El módulo reportes_page.py utiliza Matplotlib para generar gráficos circulares (pie charts) y de barras que muestran los platos más vendidos en un período seleccionable. Los reportes incluyen estadísticas de ventas totales, ingresos por categoría e informes de inventario. NumPy se utiliza para los cálculos estadísticos.



7.8 Gestión de Turnos

TurnosController registra fecha_inicio y fecha_fin para cada turno de mesero. La página empleados_page.py permite iniciar y cerrar turnos, consultar el personal activo en tiempo real y generar reportes de asistencia con horas trabajadas por empleado y período.



7.9 Menú del Día

La página menu_page.py permite al administrador configurar rápidamente los platos promocionales del día con precio especial y disponibilidad. Los platos del menú del día se destacan visualmente en la interfaz del mesero al tomar pedidos, facilitando su selección y aumentando las ventas.

```
DETALLE DE PAGO:
Plato                  Cant.  P.Unit   Subtotal   Desc.
-------------------------------------------------------
Pechuga de pollo a     2   $  2.70  $   5.40  -$1.35
-------------------------------------------------------
Subtotal:             $5.40
Propina/Servicio:     $0.54
DESC. PROMO 25%:      -$1.35
--------------------------------
TOTAL A PAGAR:        $4.59
FORMA DE PAGO:        efectivo
```

7.10 Impresión de Ticket / Factura PDF

Al completar un pago, el sistema genera automáticamente una factura en formato PDF utilizando ReportLab 4.1.0. El documento incluye: nombre del restaurante, número de mesa, fecha y hora, detalle de productos con precios unitarios, subtotal, propina aplicada, impuestos, total a pagar y método de pago utilizado.

8. Flujos Principales del Sistema

8.1 Flujo de Crear Pedido

```
1. Usuario selecciona mesa disponible  →  MesasPage
2. Click 'Nuevo Pedido'                →  PedidosPage
3. Agrega platos con cantidad          →  Selecciona plato + cantidad
4. Sistema calcula total automático    →  precio × cantidad por línea
5. Guarda en BD via PedidosController  →  Crea registro en pedidos
6. Crea DetallePedido por cada línea   →  FK: pedido_id, plato_id
7. Mesa cambia a estado OCUPADA        →  MesaEstado.OCUPADA
8. Vista de cocina se actualiza        →  Estado: PENDIENTE
```

8.2 Flujo de Procesar Pago

```
1. Pedido con estado ENTREGADO         →  PagosPage
2. Selecciona método de pago           →  EFECTIVO / TARJETA / DIGITAL
3. Sistema muestra propina sugerida    →  10% del subtotal
```

```
4. Ingresa monto / confirma pago       →  Click 'Procesar Pago'
5. PagosController valida y registra   →  Crea registro en pagos
6. Estado pedido → PAGADO              →  EstadoPago.COMPLETADO
7. Genera factura PDF                  →  ReportLab 4.1.0
8. Mesa cambia a LIBRE                 →  MesaEstado.LIBRE
```

9. Instalación y Ejecución

Requisitos previos: Python 3.8 o superior · pip (gestor de paquetes) · Sistema operativo: Windows, macOS o Linux

9.1 Pasos de Instalación

1. **Clonar el repositorio:** git clone https://github.com/Alexander200318/restaurante_mvc_pyqt6.git
2. **Crear entorno virtual:** python -m venv venv
3. **Activar entorno (Windows):** venv\Scripts\activate
4. **Activar entorno (macOS/Linux):** source venv/bin/activate
5. **Instalar dependencias:** pip install -r requirements.txt
6. **Ejecutar la aplicación:** python main.py

Para reiniciar la base de datos a su estado inicial:

```
python recrear_base.py
```

9.2 Configuración Personalizable

El archivo config.py centraliza toda la configuración del sistema y puede modificarse para adaptar el comportamiento:

```
# Base de Datos
DATABASE_PATH = Path(__file__).resolve().parent / 'restaurante.db'
DATABASE_URL  = f'sqlite:///{DATABASE_PATH}'

# Tema visual
APPEARANCE = 'dark'   # Opciones: 'dark' | 'light'

# Paleta de colores
COLORS = {
```

```
    'primary':   '#EA580C',   # Naranja
    'secondary': '#374151',   # Gris oscuro
    'success':   '#10B981',   # Verde
}
```

10. Características de Seguridad

- **Validación de Entrada:** validators.py valida todos los datos antes de persistirlos. Prevención de inyección SQL garantizada por el ORM SQLAlchemy (consultas parametrizadas automáticas).
- **Gestión de Transacciones:** Sesiones SQLAlchemy con autocommit/autoflush deshabilitado. Rollback automático ante cualquier error durante una operación de escritura.
- **Thread Safety:** StaticPool para SQLite thread-safe. check_same_thread=False configurado correctamente para entornos multi-hilo.

11. Conclusión

El sistema de gestión de restaurante desarrollado cumple con la totalidad de los requerimientos establecidos, tanto comunes como específicos del dominio, ofreciendo una solución integral y funcional para la administración completa de un restaurante.

Su diseño modular basado en el patrón MVC, su interfaz gráfica profesional con tema oscuro desarrollada en CustomTkinter, y su sistema de persistencia mediante SQLAlchemy ORM sobre SQLite, permiten mejorar significativamente la organización interna, la atención al cliente y el control del negocio.

La implementación de funcionalidades avanzadas como la comanda digital con estados progresivos, el descuento automático de ingredientes mediante relaciones M2M, la generación de facturas PDF con ReportLab, los reportes gráficos con Matplotlib y la gestión completa de turnos del personal, posicionan a este sistema como una herramienta robusta y lista para un entorno real de uso.

12. Recomendaciones para Versiones Futuras

7. **Autenticación y roles:** Sistema de login con roles (Administrador, Cajero, Mesero, Cocinero), permisos diferenciados y registro de auditoría de acciones.
8. **Base de datos remota:** Migrar de SQLite a PostgreSQL o MySQL para soporte de múltiples usuarios concurrentes y acceso desde distintos dispositivos en red.
9. **API REST:** Desarrollar endpoints REST con FastAPI o Django REST Framework para integración con aplicaciones externas y web.
10. **Aplicación móvil:** App Android/iOS para que los meseros tomen pedidos desde tablets directamente en la mesa del cliente.
11. **Sincronización en tiempo real:** Implementar WebSockets para actualización automática entre múltiples terminales simultáneas.
12. **Sistema de reservas online:** Módulo de reservas de mesas con confirmación por email/SMS y calendario de disponibilidad.
13. **Integración POS:** Conexión con lectores de código de barras, terminales de pago y cajones de efectivo físicos.
14. **Backup automático:** Copias de seguridad periódicas de la base de datos con almacenamiento en la nube.
15. **Exportación avanzada:** Exportar reportes a XML, CSV, Excel (.xlsx) y envío automático por correo electrónico.
16. **Analytics avanzado:** Dashboard de Business Intelligence con predicciones de demanda y análisis de rentabilidad por plato.

13. Referencias y Repositorios

Repositorio GitHub: https://github.com/Alexander200318/restaurante_mvc_pyqt6.git
Diseño Figma: Sistema de Restaurante — Figma Make
CustomTkinter Docs: https://github.com/TomSchimansky/CustomTkinter
SQLAlchemy Docs: https://docs.sqlalchemy.org/